

CALM: Calibration-Aware Logits Mixing for Efficient and Trustworthy Medical Image Segmentation

Research Proposal — Path A (Lab Paper Extension / Methodological Enhancement)

Md. Irtiza Hossain

M.S. Computer Science & Engineering, BRAC University
mohammad.irtiza.hossain@gmail.com

6 September 2026

Builds on LoMix (NeurIPS 2025), EMCAD (CVPR 2024), G-CASCADE (WACV 2024)
Preliminary results From my own replication of LoMix on Synapse (Deliverables 1–2)

1 Motivation and significance

Segmentation models are increasingly proposed for point-of-care use — rural clinics, bedside ultrasound, endoscopy suites — where the hardware is modest and no specialist is standing by to catch a bad contour. In that setting a model needs two properties at once. It must be **cheap at inference**, and it must **know when it is unsure**, because an overconfident wrong boundary on an organ at risk is worse than an abstention.

LoMix is a notably elegant answer to the first requirement. By supervising *mixtures* of multi-scale decoder logits under learned weights, it improves segmentation while adding **nothing** to the inference graph: the mixing weights are parameters of the loss module, not the network. I verified this directly during replication — all four supervision arms share a byte-identical inference graph (26.773 M parameters, 4.324 GMACs, 36.8 ms at 224² on a GTX 960).

But LoMix answers the second requirement not at all, and its treatment of the first is incomplete in a specific way. The learned mixing weights are **global and input-independent**: training produces one fixed answer to “which scale do I trust”, applied identically to a clean abdominal slice and to an ambiguous pancreas boundary under scanner shift. Clinically, those are not the same question. Methodologically, a supervision scheme that already reasons about *which* of its own predictions to believe is one conditioning step away from reasoning about *when* to believe them — which is the calibration question.

This proposal takes that step. **CALM** makes the mixing weights a function of per-sample uncertainty, derived from signals the decoder already computes, and adds a calibration objective alongside Dice. It keeps LoMix’s defining property — the inference graph is untouched — while addressing three concrete weaknesses I measured in the released implementation.

2 Problem definition and gaps

2.1 Gap 1 — supervision weights are static and input-independent

LoMix learns one scalar per (subset, operation) pair and freezes it after training. Under domain shift, or on the ambiguous boundaries where segmentation actually fails, the useful scale mixture plausibly differs per sample. Nothing in the formulation can express that.

My replication supplies evidence that this is not merely a theoretical concern. Starting from a common initialisation of $\text{softplus}(0) = 0.6931$, after 20 epochs the four per-scale weights had drifted to 0.379, 0.386, 0.392, 0.400 — a spread of 0.021 across scales, against a movement of

~ 0.3 away from initialisation shared by all of them. The weights moved *together*, not *apart*. Whatever differentiation the mechanism eventually achieves, at this budget it had barely begun, which suggests one global scalar per subset is a very slow-learning parameterisation for the quantity it is trying to capture.

2.2 Gap 2 — combinatorial cost falls entirely on training

For K decoder outputs and $|O|$ operations, LoMix evaluates $(2^K - K - 1) \times |O|$ mixed maps — 44 for the released $K = 4$, $|O| = 4$ configuration — each with its own CE and Dice term. Measured on identical hardware and batch:

Arm	Peak train VRAM	Step time	vs single-logit
last_layer	1008 MiB	649 ms	1.00×
deep_supervision	1054 MiB	708 ms	1.09×
mutation (unweighted mixing)	1268 MiB	938 ms	1.45×
lomix (learned weights)	2290 MiB	1649 ms	2.54×

The paper reports inference cost, which is genuinely zero. It does not report this. On constrained hardware, 2.54× training cost decides whether the method can be used at all — and it scales as 2^K , so deeper decoders make it worse.

2.3 Gap 3 — the objective is accuracy-only

LoMix optimises CE + Dice. Neither is a calibration objective, and the paper reports no calibration metric. A method whose entire premise is *weighting predictions by how much they should be trusted* never measures whether the resulting confidence is trustworthy.

2.4 Gap 4 — two of the four mixing operations are defective as released

Found and verified during replication (`tools/diagnose_loss_ops.py`).

mul is numerically unbounded. It multiplies raw logits, so a k -map combination is a k -fold product growing like $|\text{logit}|^k$. On the trained checkpoint’s own logits ($\max |p| = 6.16$):

Operation	$k = 2$	$k = 3$	$k = 4$
add	12.0	17.9	23.6
mul	36.2	211.0	1214.0
wf	6.1	6.1	5.9
concat	2.8	2.9	1.9

In my run this diverged to NaN at epoch 13 under the paper’s own hyperparameters, destroying the arm.

concat cannot learn. Its 1×1 fusion convolution is constructed *inside forward*, randomly re-initialised on every call, never registered as a submodule and never given to the optimizer. Two identical forward passes under `no_grad` differ by 5.41 for `concat` and by exactly 0.0 for `add`, `mul` and `wf`. Consistent with this, `concat`’s eleven learned weights collapsed to a common 0.3755–0.3770 and stopped moving — there is no gradient signal for them to differentiate on.

So of four mixing operations, one injects unbounded values and one injects random noise. Any claim about *which* mixtures matter rests on an operation set that is half broken.

2.5 Related work and what is missing

Deep supervision and multi-scale fusion (UNet++, CASCADE, EMCAD, G-CASCADE, LoMix) weight scales by fixed or globally-learned coefficients. None conditions on the input.

Uncertainty estimation (MC dropout, deep ensembles, evidential segmentation) produces per-sample uncertainty but at multiplied inference cost — the opposite of what deployment on a point-of-care device needs.

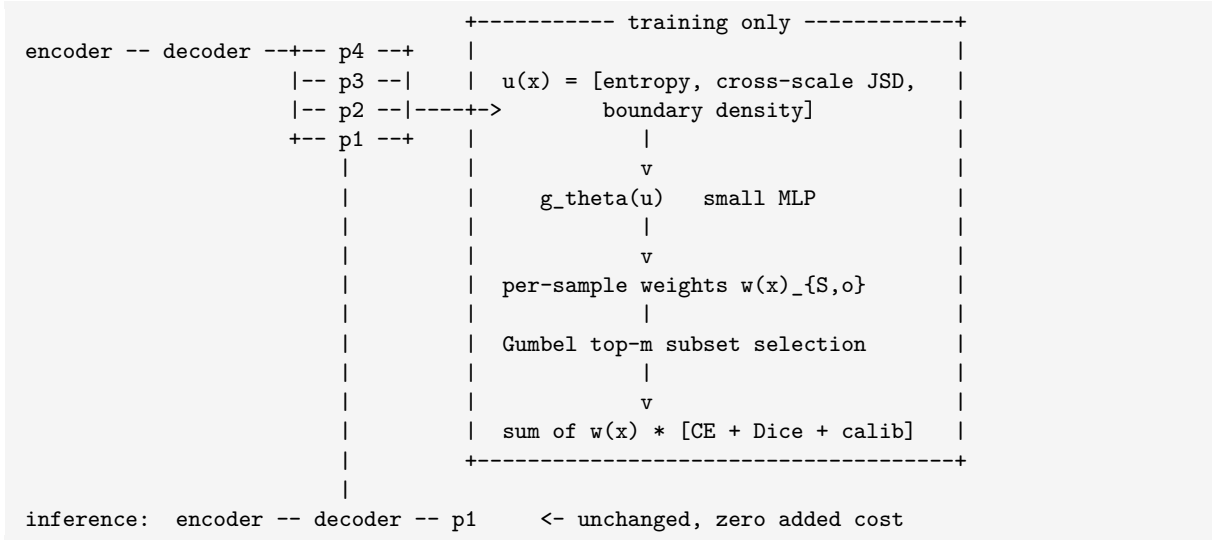
Calibration (temperature scaling, focal loss, proper scoring rules) is applied post-hoc or as a loss term, but is not connected to *how supervision is allocated across scales*.

Conditional computation / routing (mixture-of-experts, dynamic networks) conditions computation on the input, but in the network, so inference cost rises.

The gap CALM occupies: **per-sample conditioning of the supervision, not the network**. Because the conditioning lives entirely in the loss, it costs nothing at inference — which no uncertainty-aware or conditional-computation method above can say.

3 Proposed method

3.1 Overview



3.2 Bounded mixing operations (prerequisite)

Before conditioning anything, the operation set must be sound.

- **mul in probability space.** Multiply per-class softmax probabilities and renormalise (equivalently, *average logits* — a log-domain product), which is a principled product-of-experts fusion and is bounded by construction. This removes the divergence mode in §2.4 at no representational cost.
- **concat registered properly.** Move the 1×1 fusion convolutions into an `nn.ModuleDict` keyed by subset size, as the `wf` operation already correctly does. This makes a quarter of the operation set functional for the first time.

Both changes carry falsifiable predictions: `mul` should stop diverging, and `concat`'s learned weights should move away from their collapsed value. If `concat` remains at the collapsed value once trainable, that is itself an informative result about which mixtures matter.

3.3 A free per-sample uncertainty signal

The key design constraint is that uncertainty must not cost an extra forward pass, or the efficiency argument collapses. CALM therefore derives $u(x)$ from quantities the decoder has already produced:

1. **Predictive entropy** of the final map, mean over foreground pixels.
2. **Cross-scale disagreement** — mean Jensen–Shannon divergence between the softmax outputs of the four decoder scales. This is exactly the signal that should govern how much to trust each scale, and it is available for free because all four maps are already computed for the loss.
3. **Boundary density** — a cheap proxy for the ambiguous-boundary regime, from the gradient magnitude of the predicted mask.

Cross-scale disagreement is the conceptually important one: it makes the weighting responsive to *the specific failure mode the mixing is meant to fix*.

3.4 Conditioning the weights

Replace LoMix’s global scalar $w_{S,o}$ with

$$w(x)_{S,o} = \text{softplus}\left(a_{S,o} + b_{S,o}^\top g_\theta(u(x))\right)$$

where $a_{S,o}$ is the original global parameter (so CALM strictly generalises LoMix, recovering it exactly when $b = 0$), g_θ is a small MLP ($3 \rightarrow 16 \rightarrow 8$), and $b_{S,o}$ is a learned per-term projection. Total added parameters $\approx 10^3$, all in the loss module, none at inference. Initialising $b = 0$ means training starts exactly at LoMix and departs only if conditioning helps — which makes the comparison against the baseline clean rather than confounded by initialisation.

3.5 Uncertainty-guided subset selection

Rather than evaluating all 44 mixed maps every step, sample $m \ll 44$ subsets per step via Gumbel top- m over the conditioned weights, with a straight-through estimator. Terms the model considers uninformative for *this* sample are skipped. This attacks §2.2 directly and is the mechanism by which CALM aims to be *cheaper* than LoMix in training, not merely better. Expected cost at $m = 12$ is roughly $1.4\text{--}1.6\times$ the single-logit baseline versus LoMix’s measured $2.54\times$, to be confirmed empirically.

3.6 Calibration in the objective

Add a differentiable calibration term to each supervised map — a soft-binned ECE surrogate or a proper scoring rule — weighted by λ . Report ECE and reliability diagrams alongside Dice. This turns “which predictions to trust” from an implicit assumption into a measured quantity.

4 Experimental plan

Datasets. Synapse multi-organ and ACDC (in-distribution, directly comparable to the lab’s published tables); polyp datasets (Kvasir-SEG, CVC-ClinicDB, CVC-ColonDB, ETIS) for a **cross-centre domain-shift** protocol — train on two centres, test on the held-out ones, which is where per-sample conditioning should matter most and where a static global weight should not.

Baselines. `last_layer`, `deep_supervision`, `mutation`, `lomix` (as released), `lomix + bounded ops`, and CALM ablations: `bounded-ops only`, `+conditioning`, `+selection`, `+calibration term`.

Metrics. Dice and HD95; **ECE, adaptive ECE, reliability diagrams, AURC**; and training cost (step time, peak memory) measured under identical conditions, since a cost claim is part of the contribution.

Key ablations. (i) Which uncertainty feature carries the signal — entropy alone vs cross-scale JSD alone vs all three. (ii) Sensitivity to m . (iii) Does conditioning help more under shift than in-distribution — the central prediction. (iv) Do the conditioned weights vary meaningfully across samples, or collapse back to a global solution? A negative here would falsify the premise, and I would report it.

Compute. In-distribution Synapse/ACDC runs are ~ 7 h per arm at reduced budget on the hardware I have used so far; the full protocol needs cluster access, which I would expect to plan around rather than assume.

5 Preliminary results

All from my own LoMix replication (Deliverable 1); every figure traces to a run log. Reduced budget: 20 of the paper’s 300 epochs, effective batch 6, seed 2222.

Arm	Best mean Dice	Status
last_layer	0.6764	completed
deep_supervision	0.7979	completed
mutation	0.7788	completed
lomix	0.7225 (pre-divergence)	diverged (NaN) at epoch 13

These runs do not reproduce the paper’s ordering, and I do not claim they refute it. At 20 of 300 epochs, with one seed, the mixing weights had barely differentiated (§2.1) — the mechanism that is supposed to produce LoMix’s advantage had not yet engaged. What the runs *do* establish, independently of budget, is: the zero-inference-overhead claim (measured directly), the $2.54\times$ training-cost ratio (measured under identical conditions), and the two implementation defects in §2.4 (verified on the trained model’s own logits, not dependent on training length).

Those three facts are precisely the premises this proposal builds on, which is why the replication was worth doing before proposing anything.

6 Expected outcomes, risks and timeline

Expected outcomes. (1) A supervision scheme that conditions on per-sample uncertainty at zero inference cost and *lower* training cost than LoMix. (2) Improved calibration under domain shift at equal or better Dice. (3) A corrected, properly-registered operation set, with an empirical account of which mixtures actually matter once all four operations work.

Risks and mitigations.

- *Uncertainty estimates are themselves miscalibrated early in training.* Mitigate with a warm-up during which $b = 0$ (pure LoMix), enabling conditioning only once predictions are meaningful.
- *Conditioning overfits the training distribution.* Mitigate by keeping g_θ deliberately tiny and evaluating primarily on held-out centres.
- *Gumbel selection adds gradient variance.* Mitigate by annealing m downward from 44, so selection is introduced only after the weights are informative.
- *The premise is wrong and per-sample weights collapse to a global solution.* This is a real possibility. It is also a publishable negative result about the structure of multi-scale supervision, and the ablation in §4(iv) is designed to detect it rather than hide it.

Timeline (first 12 months). Months 1–3: bounded operations, corrected concat, reproduce LoMix at full budget as the reference point. Months 4–6: conditioning and calibration objective; in-distribution results on Synapse/ACDC. Months 7–9: uncertainty-guided selection and the cost study; cross-centre polyp protocol. Months 10–12: ablations, writing, submission (MICCAI or ISBI, with the efficiency study a natural fit for a CVPR/ICCV workshop).

7 Fit with the group

The proposal extends a lab paper along the lab’s own axis — efficiency without accuracy loss — and adds the trustworthiness dimension the group’s stated direction emphasises. It reuses the existing LoMix/EMCAD codebase rather than starting elsewhere, and the first three months produce something useful to the group regardless of whether the main hypothesis holds: a corrected operation set and a full-budget reference run.

It also matches what I have been doing. **CLEAR-MoE** converts frozen Vision Transformers into sparse mixture-of-experts models using a calibration-driven layer-selection criterion — the same “use calibration to decide where to spend capacity” idea, applied to architecture rather than supervision. **Predictive Self-Conditioning** conditions a LiDAR detector on its own predictions and is evaluated on calibration precisely because self-conditioning compounds error when confidence is wrong. CALM is the same question in the lab’s domain: when a model weights its own multi-scale predictions, that weighting should depend on how much those predictions deserve to be trusted for *this* input.